MEMORIA TÉCNICA - «Más joven que tú»

Documentación técnica de la **Base 6** del Decreto Foral 73/2026: procedencia y forma de acceso de los datos utilizados, descripción del proceso de trabajo con los datos y herramientas/técnicas empleadas.

Revisión: **07-11-2026**. Web pública: https://masjovenquetu.eus (copia permanente: https://huntsman1756.github.io/mas_joven_que_tu-/). Identidad, CI y QA de la publicación: evidence/vps2-eus-20261107/SUMMARY.md (servidor propio, ADR-029) y evidence/pages-20261107/. Capturas y demo recapturadas el 07-11-2026 desde la web publicada (build 32e4831). Desde la publicación del 01-10-2026 (fuente 469b989) los cambios son de interfaz, copy y despliegue; no cambian datos, pipeline ni métricas (git diff 469b989.. -- data pipeline app/static/data vacío). El paquete vigente es output/pdf/final-20261107-v2/; su manifiesto identifica cada archivo y el sello del build. Los paquetes anteriores son históricos. Persisten las pruebas humanas enumeradas en §7 y la solicitud administrativa.

1. Qué es el producto

Pieza de periodismo de datos estática y explorable: el visitante indica su año de nacimiento y su municipio, y el producto responde qué proporción de los edificios **que hoy existen** en ese municipio es posterior a ese año.

A partir de la respuesta, **cinco modos de exploración** comparten el mismo estado (año, lugar, cámara y capas):

Modo	Pregunta
Por antigüedad (mapa)	Qué edificios actuales son posteriores a mi año; cuota por zona de 500 m y detalle por edificio
Evolución	Cómo se acumulan los años de construcción del stock superviviente (cabezal + reproducción)
Fotos aéreas	Qué muestra un vuelo oficial del lugar (campañas discretas, opt-in de red)
Mapa 1923-25	Qué muestra la cartografía histórica (raster georreferenciado, distinto de foto y dato)
Antes/ahora	Comparar la evidencia visual de dos campañas con cortina arrastrable

Además: **cinco capítulos editoriales** (casos concretos con universo propio), búsqueda de dirección, contexto de población y planeamiento vigente, y la página «Cómo lo sabemos» con método, fuentes, licencias y límites.

El ejemplo de Mungia abre el hallazgo junto al mapa. El conjunto de 70 edificios se identifica en el encabezado; el municipio completo tiene un bloque plegable de contexto separado. Las cifras locales no se presentan como cifras municipales.

La entrada en **Fotos aéreas** no descarga imagen alguna hasta una activación explícita: si el visitante no ha elegido campaña, el panel lo dice y ofrece la acción (0 peticiones de ortofoto antes del opt-in, verificado por sonda).

Afirmación semántica central (no negociable, docs/DATA_SEMANTICS.md): los datos describen el **parque de edificios existente hoy**, no una reconstrucción del parque histórico. El producto ordena los edificios actuales por su año registrado; nunca afirma cuántos edificios existían en una fecha pasada.

2. Datos utilizados: procedencia y acceso

2.1 Fuente principal - Open Data Bizkaia (Base 1)

Dataset	Procedencia / acceso	Uso
Parcelario catastral - edificios (112 municipios)	opengis.bizkaia.eus/.../Open Data/{COD}_{MUNI}_{GML|SHP}.zip; catálogo opendatabizkaia.eus/es/catalogo/parcelario-catastral	Capa Edificio: geometría + Ano_Constr. Única fuente del año de construcción
Ortoimágenes 1956-2002 (9 campañas: 1956, 1965, 1970, 1975, 1983, 1990, 1995, 1999, 2002)	Tiles opengis.bizkaia.eus/.../MapServer/tile/{z}/{y}/{x}; recortes locales de portada documentados en sus manifests	Evidencia visual; consulta remota del visor con activación opt-in
Cartografía histórica 1923-1925	opengis.bizkaia.eus/.../ORTO_EJ_CARTO_1925/... (servicio de teselas)	Mapa histórico standalone, opt-in
Límites municipales	Open Data Bizkaia	Geometrías de municipio
Planeamiento urbanístico	Open Data Bizkaia (snapshot municipal)	Contexto de capacidad registrada
Contexto territorial (montes, ruido, bizkaibus…)	Open Data Bizkaia	Hechos de lugar con denominador de solape

2.2 Fuentes complementarias (explícitamente no principales)

Dataset	Procedencia / acceso	Uso
Ortofotos **1945, 1977, 1984, 1989, 1991, 2001 y 2004-2025** (28 campañas)	WMS WMS_ORTOARGAZKIAK de geoEuskadi (Gobierno Vasco, CC BY 4.0)	Completa la serie fotográfica donde termina Open Data Bizkaia (incluye el vuelo americano 1945)
NORA geocoder	Servicio oficial Gobierno Vasco (CORS *)	Búsqueda de dirección; sin clave, sin tracking
Población municipal	API PXWeb de Eustat (eustat.eus), snapshot propio app/static/data/eustat-population.json	Contexto humano «Qué más sabemos del lugar»
Callejero municipal (capa de portales)	Callejero de la CAPV en Open Data Euskadi / Eustat (CSV, CC BY 4.0)	Sugerencias de calle en «Mi edificio»; ver ADR-020

El catálogo de campañas con su año **nominal**, rango de vuelo cuando la fuente lo publica, licencia y previsualización está en app/static/data/catalog.json (37 campañas; regenerable con python pipeline/g1_buildings.py --catalog-only).

Licencias: recursos de Bizkaia, geoEuskadi y el callejero de Open Data Euskadi según sus declaraciones CC BY 4.0; Eustat permite redifusión con cita conforme a su aviso legal. No se asigna automáticamente CC BY a una fuente diferente. El detalle por dataset está en docs/submission/SOURCES-LICENSES.md y en data/manifests/*.yaml (un manifiesto por fuente: URL, fecha de descarga, licencia, CRS, limitaciones conocidas).

3. Proceso de trabajo con los datos

3.1 Obtención y congelación

1. Descubrimiento vía API CKAN de Open Data Bizkaia (package_search), lectura de access_URL y licencia por recurso.

2. **Preingesta** (python pipeline/g0_recon.py): descarga y extracción de los ZIP SHP/GML por municipio hacia data/interim/catastro/<cod>/ (gitignored) y registro de **112 descargas con SHA-256, bytes y URL** en evidence/g0/02-recon/recon-bizkaia.json. Fallos declarados en failures (en este corte: 0).

- No confundir con data/snapshots/catastro_manifest_20260918.json: es el **inventario del listado** del portal (176 URLs = SHP + GML) con un digest de listado, **sin** hash por fichero. Ver data/manifests/README.md.

- Los ZIP crudos no se versionan por tamaño: conservar data/raw/ o aceptar que una regeneración posterior descargue una fuente viva.

3. Cada ejecución registra snapshot de origen, transformación aplicada, informe de QA y fecha (data/qa/). Sin excepciones.

3.2 Análisis y contratos de métrica

- Una sola consulta DuckDB Spatial (ST_Read sobre SHP/GML) produce por municipio: total de edificios (C-01), con año válido (C-02), desconocidos/sospechosos/inválidos, huella en planta aceptada (C-06) y la serie acumulada por año (cum_buildings, cum_footprint_area).

- El año Ano_Constr se clasifica VALID | UNKNOWN | SUSPICIOUS | INVALID (§5 de DATA_SEMANTICS). UNKNOWN ≠ 0: un 0 o vacío nunca se convierte en año ni en dato. 139.447 edificios actuales (C-01); 138.501 con año válido (C-02, cobertura 99,32 %); 936 sospechosos.

- Las métricas siguen contratos con universo, numerador y denominador (§11 de DATA_SEMANTICS); pipeline y frontend usan el mismo denominador.

- Área derivada de polígono = **huella en planta**, nunca superficie construida total. Reparaciones geométricas (ST_MakeValid) trazadas con original, transformación, motivo y resultado.

- El año nominal de campaña de ortofoto se muestra junto a la fecha real de vuelo cuando se conoce; la cartografía 1923-25 se presenta como mapa, nunca como fotografía ni como fecha de construcción.

3.3 Publicación estática

- Vector tiles PMTiles (tippecanoe, contenedor fijado - ADR-003): cells.pmtiles (celdas con cuota de edificios posteriores al año), municipalities.pmtiles, índices por edificio.

- JSON por municipio en app/static/data/ (métricas, planeamiento, contexto, población Eustat). El cliente consulta acumulados y calcula las cuotas del año elegido conforme a los mismos contratos del pipeline.

- Ortofotos e histórico del visor se sirven desde los servicios oficiales tras activación explícita (incluida una URL con campaña seleccionada). La portada y miniaturas utilizan recortes locales, no peticiones al servicio externo.

4. Herramientas y técnicas

Pieza	Herramienta	Licencia
Pipeline de datos	Python + DuckDB (ext. spatial)	MIT
Vector tiles	tippecanoe (fork felt) en contenedor fijado	BSD-2
Frontend	SvelteKit (adapter-static) + Svelte 5 + TypeScript	MIT
Mapa	MapLibre GL JS + PMTiles	BSD-3
Comparación fotográfica	Modo «Antes/ahora»: dos mapas sincronizados (el principal y el lienzo superpuesto recortado por la cortina arrastrable) con estados de cobertura por lado; pointer + teclado (ADR-016, SwipeCompare.svelte, map/sync.ts)	-
Verificación	Playwright (sondas E2E multi-ruta), Vitest, axe-core, ESLint, Prettier	MIT/Apache
Servidor estático	Node con soporte HTTP Range (PMTiles lo exige)	-

Inventario completo de terceros y decisiones ADOPT/REJECT: docs/OSS_REUSE.md. Sin backend: el producto es un build estático.

5. Reproducibilidad

Estado: PARCIAL (no FULLY_REPRODUCIBLE, no NOT_REPRODUCIBLE):

- **Frontend**: sí - npm ci && npm run build con app/package-lock.json fija todas las dependencias.

- **Datos**: parcial - los artefactos publicados están en el repo, pero los ZIP crudos de Catastro no se versionan; regenerar desde cero requiere pipeline/g0_recon.py contra la fuente viva (o los ZIP conservados localmente) y entonces el resultado es un **corte nuevo**, no el snapshot publicado. Reproducir el snapshot exige exactamente esos bytes (112 SHA-256 en evidence/g0/02-recon/recon-bizkaia.json).

- **Servicios en runtime**: las ortofotos y cartografías oficiales se consultan en vivo; su disponibilidad no depende del repo.

Receta verificable (también en AGENTS.md §Comandos):

powershell -File scripts\verify.ps1 # check + eslint + format:check + tests + build + pytest + Range

python pipeline\g0_recon.py # preingesta (solo si hay que regenerar datos)

python pipeline\g1_buildings.py --only 020,054,908 # subset de humo (CLI real)

bash scripts/g1_build_tiles.sh # tiles (contenedor fijado)

Versiones efectivas medidas en este entorno: Node 24.19.0 / npm 11.17.0 (CI: Node 20), Python 3.11.15 con duckdb 1.5.5, requests 2.34.2, shapely 2.1.2; Playwright 1.63.0; tippecanoe 2.79.0 (contenedor). pipeline/requirements.txt fija rangos; pipeline/constraints-release.txt congela el entorno comprobado del candidato (instalación con pip install -r pipeline/requirements.txt -c pipeline/constraints-release.txt). El lock de npm fija el frontend. No se afirma reproducción bit a bit del build de SvelteKit.

Cada fuente tiene manifiesto (data/manifests/) y QA (data/qa/); las cifras visibles del producto se derivan de esos artefactos, no de cálculos ad hoc. Las sondas de navegador verifican semántica en ejecución: 0 peticiones de ortofoto sin opt-in, UNKNOWN nunca renderiza como 0, el mapa histórico falla cerrado, la línea temporal solo muestra el eje catastral, cobertura y denominadores visibles.

6. Identificación del build candidato

- El HTML servido lleva <meta name="mjt:build" content="<sha>"> (o <sha>+dirty(n) si el árbol tiene cambios sin commitear), escrito por app/scripts/seo-static-head.mjs en cada npm run build: la atribución build↔commit no depende del mensaje del commit de Pages.

- El build de producción usa rutas relativas; se verifica servido bajo el prefijo real de GitHub Pages (/mas_joven_que_tu-). Range/PMTiles se comprueba sobre app/build, no se infiere de una respuesta HTML 200.

- Procedimiento de publicación y rollback: docs/remediation/red-team-2026/RELEASE.md.

7. Incertidumbre declarada (visible en producto)

- Cobertura de año por municipio mostrada junto a cada cifra.

- Edificios «sin año utilizable» con canal visual propio (hatch), nunca asimilados a un año.

- Año anómalo contado aparte («Otros N registran un año anómalo»).

- Cifras redondeadas sin fingir exactitud: extremos de cuota como «<0,1 %» / «>99,9 %» y sin «cifra exacta» sobre un redondeo.

- Planeamiento = capacidad registrada hoy; no uso histórico ni predicción. Contexto = solape/proximidad actual; nunca causalidad.

- EU: estructura completa (paridad de claves y placeholders verificados por test) con **revisión lingüística nativa pendiente**; la paridad de claves no certifica la traducción. Lista de claves/contextos: docs/remediation/red-team-2026/EU_NATIVE_REVIEW.md.